САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Фронт-энд разработка

Отчет

Лабораторная работа 2: Взаимодействие с внешним API
Тема: Платформа для покупки билетов на мероприятия

Выполнил:

Эль-Сибаи Амир

Группа J3211, ИСУ 468154

Проверил:
Добряков Д. И.

Санкт-Петербург

2026 г.

Идея лабораторной работы:

UCL Tickets — веб-платформа для покупки и продажи билетов на матчи Лиги Чемпионов УЕФА сезона 2025/26. На данный момент матчи на стадии четвертьфиналов.

В рамках второй лабораторной работы сайт был переведен на динамическую работу с данными. Реализован моковый REST API с использованием пакета json-server. Все данные о пользователях, матчах, купленных билетах и выставленных на продажу лотах загружаются, создаются и изменяются через асинхронные HTTP-запросы (fetch).

Работа с API:

1. Создание нового пользователя и вход в систему с проверкой данных в базе с сохранением сессии в localStorage.
2. Подгрузка списка предстоящих матчей на страницу каталога и в выпадающие списки.
3. Формирование объекта заказа и отправка его на сервер с автоматическим пересчетом итоговой стоимости.
4. Изменение личных данных и смена пароля.
5. Изменение статуса купленного билета в базе данных.
6. Возможность стать организатором для получения доступа к функционалу продаж.
7. Создание новых объявлений о продаже билетов.

Файловая структура:

К структуре из первой лабораторной работы добавился файл базы данных и полностью переписан скрипт:

- **db.json** — моковая база данных (JSON Server), хранящая коллекции: users, matches, tickets, listings, reviews.
- **script.js** — единый JavaScript-файл, обрабатывающий логику приложения.
- HTML/CSS файлы — пользовательский интерфейс.

Ход работы

1. Настройка базы данных и сервера

   С помощью утилиты json-server был запущен локальный сервер, имитирующий работу backend-приложения. Данные о расписании матчей и отзывах были занесены в db.json заранее. Таблицы пользователей и билетов оставлены пустыми для заполнения через интерфейс.

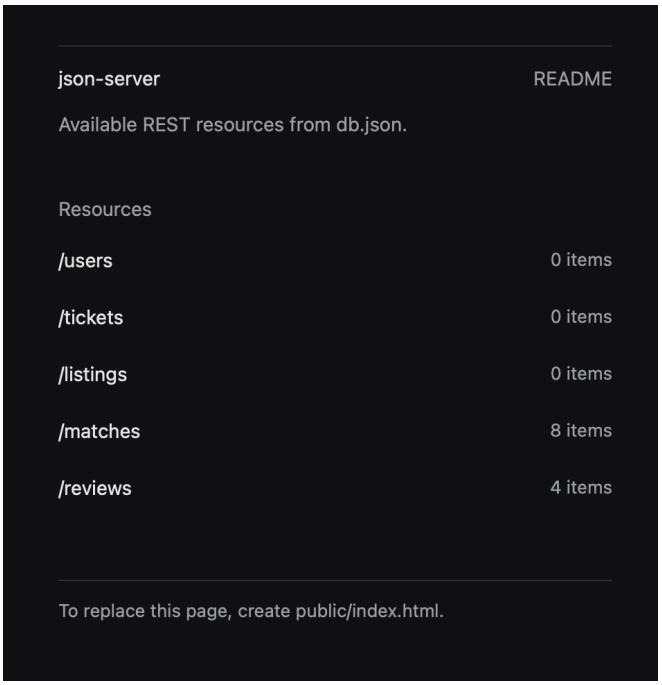
```
(base) mir@MacBook-Pro lab2 % npx json-server --watch db.json --port 3000
--watch/-w can be omitted, JSON Server 1+ watches for file changes by default
JSON Server started on PORT :3000
Press CTRL-C to stop
Watching db.json...

♡ᐟ(ᐟ> ˘ <ᐟ)ᐟ♡

Index:
http://localhost:3000/ Start Debugging in Browser

Static files:
Serving ./public directory if it exists

Endpoints:
http://localhost:3000/users Start Debugging in Browser
http://localhost:3000/tickets Start Debugging in Browser
http://localhost:3000/listings Start Debugging in Browser
http://localhost:3000/matches Start Debugging in Browser
http://localhost:3000/reviews Start Debugging in Browser
```



2. Регистрация и авторизация

   При регистрации скрипт собирает данные из формы, проверяет совпадение паролей и отправляет POST-запрос на /users. При входе в систему выполняется GET-запрос для получения списка пользователей, после чего на стороне клиента происходит поиск совпадений по email и password. Если пользователь найден, его данные сохраняются в localStorage.

3. Динамический вывод каталога билетов и фильтрация

   На странице tickets.html при загрузке страницы отправляется GET-запрос к /matches. На основе полученного массива JSON генерируются HTML-элементы карточек, хранящихся в БД.

4. Страница мероприятия и покупка

   При переходе на конкретный матч, ID матча передается через URL-параметр ?match=. Скрипт делает GET-запрос конкретного объекта и заполняет страницу. При нажатии "Confirm Purchase" формируется объект с данными о билете, который отправляется POST-запросом в коллекцию /tickets.

5. Личный кабинет и возвраты

   В личном кабинете отправляется запрос к /tickets?userId=... для получения билетов конкретного пользователя. При нажатии на кнопку "Refund" отправляется PATCH-запрос, который меняет статус билета в базе с Active на Refunded. В этом же разделе реализовано обновление личных данных и пароля.

6. Функционал организатора

   Если пользователь нажимает "Become Organizer", его роль в db.json меняется. В интерфейсе появляется вкладка "Sell Tickets". В этой вкладке выпадающий список матчей заполняется через API. Организатор может создать объявление или снять билет с продажи.